# A Human-Centered Architecture for Instrumented Virtual Reality Training with Digital-Twin Connectivity

Anonymous Author(s)
*Submitted for double-blind review*

***Abstract*—Virtual reality can support industrial training, but a realistic scene alone cannot control a procedure, explain trainee actions, or produce reusable evidence. This paper presents an instrumented immersive platform for a labeling workstation. Implemented in Unity 6 for Meta Quest 3, the platform uses a configurable state machine to link each task step with guidance, completion rules, interaction validation, and telemetry. A separate ESP32–FastAPI pathway delivers a machine measurement while keeping training independent of network availability. Technical evaluation covered requirement traceability, runtime inspection, offline execution, and 27 session records. Records contained total duration, nine step times, and nine error counters. The system captured 104 invalid-interaction events, while the mean absolute difference between total and step time was 0.445 s. These results demonstrate end-to-end interaction traceability and identify a boundary-timing limitation. The contribution is a modular architecture combining procedural control, explainable validation, objective logging, and bounded physical-to-digital connectivity while preserving trainee autonomy.**

***Index Terms*—virtual reality, smart factory, procedural training, digital twin, state machine, interaction validation, telemetry**

## I. Introduction

Smart manufacturing changes the knowledge required from human operators. In addition to understanding a production sequence, an operator may need to interpret digital guidance, manipulate physical resources, and respond to changes in equipment state. Conventional demonstrations and written procedures remain useful, but they are difficult to repeat under identical conditions and normally provide little information about where an execution failed or became uncertain. The engineering objective is not to automate the person out of the loop. It is to create a place where an operator can rehearse, make recoverable mistakes, and receive feedback before those mistakes carry physical consequences.

Immersive virtual reality (VR) can reproduce an industrial workspace without interrupting production or exposing trainees to equipment hazards. However, a rendered three-dimensional scene is not by itself a training system. A technically useful platform must represent the procedure explicitly, constrain or interpret interaction, vary assistance, and record what happened during execution. Digital-twin research adds a second requirement: the virtual environment should have a defined data boundary with the physical system instead of being described as a twin solely because it resembles a factory [1], [2].

This paper addresses the following research question: *How can a human-operated smart-factory procedure be represented as a modular immersive system that preserves operator agency while providing deterministic task control, explainable validation, objective telemetry, and bounded access to connected factory data?* The implementation is grounded in a manual labeling workstation within a laboratory-scale smart-factory line. The workstation was reconstructed in Unity 6 and deployed on Meta Quest 3.

The paper makes four contributions:

- a requirement-driven architecture separating interaction, procedural control, guidance, logging, and supervision;
- a state-machine model that makes the relation between an operator action, its validation rule, and its recorded evidence explicit;
- a guidance and error-handling strategy that treats assistance as configurable support and invalid actions as recoverable learning events; and
- a bounded ESP32–FastAPI–Unity connection that adds factory context without making training dependent on connectivity.

The scientific claim is deliberately architectural: the artifact realizes and demonstrates a traceable set of design requirements. Existing session data are used to verify observability, not to infer a training effect.

## II. Related Work

### A. Immersive Training for Manufacturing

Recent reviews identify extended reality (XR) as a useful medium for manufacturing instruction, assembly support, maintenance, and operator familiarization [3], [4]. Immersive systems permit safe repetition and embodied interaction, while the combination of spatial cues and direct manipulation can support procedural understanding [5], [6]. Reviews of virtual training also report substantial variation in study design, session reporting, and outcome measures [7]. This variation makes the internal control and instrumentation architecture important when a training environment must support reproducible scenarios.

The quality of an XR experience also depends on whether system responses are congruent with user actions and plausible within the represented task [8]. In an industrial procedure, congruence requires more than realistic graphics: an object

should be selectable, placeable, and valid only when those operations are meaningful for the current task state. The present work therefore treats expected-action validation as a first-class runtime component.

### B. Digital Twins and Human-Centered Manufacturing

Digital twins are commonly characterized by an identifiable physical entity, a virtual representation, and data connections supporting a defined lifecycle purpose [1], [9]. Modeling, synchronization, communication, and human interaction are therefore central concerns rather than optional implementation details [10], [11]. ISO 23247 further structures manufacturing digital twins around observable manufacturing elements and information exchange [12]. Smart-manufacturing reference models similarly emphasize the link between physical resources, data services, and application behavior [13].

The current platform does not claim closed-loop control of the production line. It implements a unidirectional supervision proof of concept and maintains a separate offline training scenario. This distinction avoids conflating a static virtual model, a digital shadow, and a bidirectional operational twin. It also supports a human-centered design: the connected data provide context, while the procedure manager remains responsible for guidance and evaluation. This focus is consistent with recent work on ethical technology design, human-centric manufacturing, and human-cyber-physical systems [14]–[18].

### C. System Gap

Prior work establishes the educational potential of VR and the data requirements of digital twins. The engineering gap addressed here is their runtime integration at workstation scale: procedure definitions, XR events, validation rules, guidance policies, objective logs, and connected measurements must share explicit interfaces. The proposed architecture makes these interfaces visible, testable, and independently extensible.

### D. Design-Science Method

The work follows an artifact-oriented design-science sequence [19]. First, the physical labeling workflow was observed and decomposed into operator goals, objects, actions, and completion conditions. Second, these observations were translated into six system requirements. Third, the architecture was implemented as a Unity artifact. Finally, runtime traces and the connected-data proof of concept were inspected against the requirements. This method evaluates whether the proposed mechanisms are present and operational; it does not substitute for a controlled study of learning outcomes.

## III. Context and Design Requirements

### A. Labeling Workstation

The case study uses a laboratory-scale line representing the production and processing of high-density polyethylene bottles through extrusion-blow molding, trimming, filling, capping, labeling, quality control, packaging, and storage. Labeling is a manual workstation and therefore provides a compact but representative human-in-the-loop procedure.

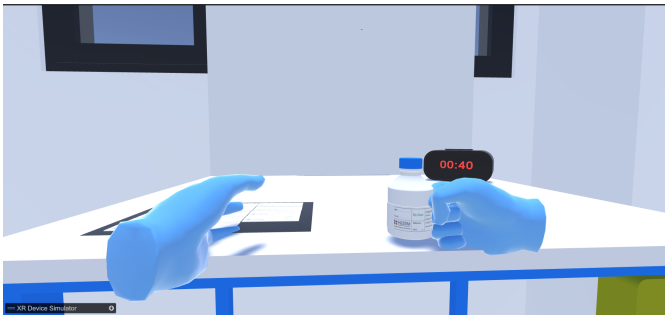


Fig. 1. Operator view of the immersive labeling workstation. The virtual hands, bottle, label, timer, and spatial work area form the human-facing layer of the system.

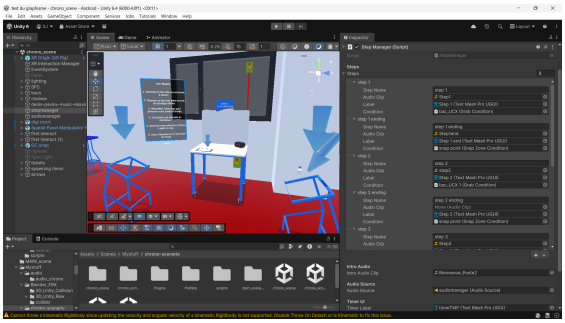

Fig. 2. Unity editor configuration of the procedural controller. States expose their instruction, audio cue, interface label, and completion condition to the scenario designer.

The user-facing operation contains six principal actions: positioning an empty bin, placing it in the designated zone, supplying the workstation with a second bin, obtaining a label, selecting a bottle, and applying the label to the correct face. The Unity implementation decomposes pickup and placement checks into nine internal states. This finer decomposition permits the system to time and validate transitions that are visually presented to the trainee as one operation.

### B. Requirements

Table I summarizes the requirements derived from the workstation and intended users. Together they define a traceability chain from procedure to interaction evidence while keeping guidance supportive rather than coercive.

## IV. Platform Architecture

### A. Layered Organization

Fig. 3 presents five layers with explicit responsibilities. The device layer provides XR input and optional physical measurements. Mediation isolates device protocols from the application. The interaction layer converts low-level input into task events. Procedural services interpret those events, while the evidence layer stores execution traces. The training loop depends on the XR path but not on the optional physical-data path.

TABLE I
PLATFORM REQUIREMENTS

| ID | Requirement | Engineering implication |
|---|---|---|
| R1 | Procedural traceability | A controller owns the active state and legal transition. |
| R2 | Explainable validation | Grabs and placements are checked by explicit task rules. |
| R3 | Graduated guidance | Audio, text, and spatial cues can be reduced without changing the procedure. |
| R4 | Publication-safe evidence | Shared task records exclude direct identifiers. |
| R5 | Graceful degradation | Training remains operational without a factory connection. |
| R6 | Bounded connectivity | Physical data enter Unity through a read-only mediation layer. |

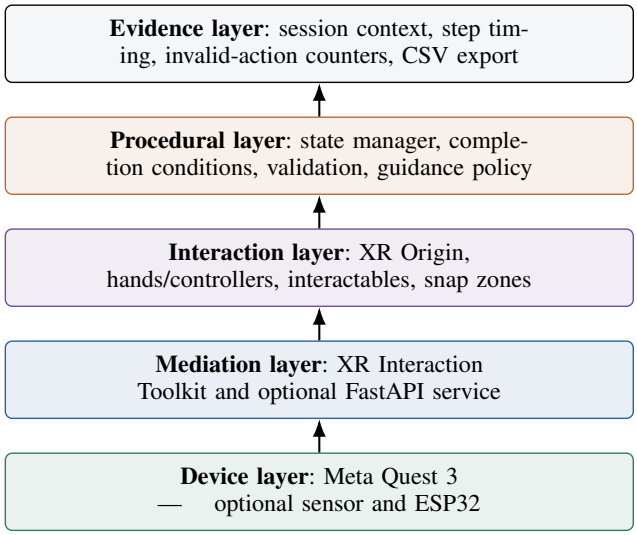


Fig. 3. Layered architecture and responsibility boundaries. Physical sensing is an optional branch within the device and mediation layers; the XR training path remains operational without it.

### B. XR Interaction Layer

The Unity 6 scene uses the XR Interaction Toolkit and contains an XR Origin, an XR Interaction Manager, interactive bottles and bins, snap zones, spatial panels, audio sources, and task-specific animation. Controller or hand interaction is converted into selection and placement events by the XR layer. The procedure controller does not interpret low-level pose data; it receives semantically meaningful events such as "object grabbed" or "object entered target zone." This boundary follows the portability goal of OpenXR, which provides a common application interface across conformant XR runtimes [20].

Teleportation and turning support movement in the scene, but locomotion does not advance the procedure. State progression occurs only when a registered completion condition is satisfied. Consequently, exploration and accidental contact can be distinguished from task completion.

### C. Procedural State Machine

Let the internal task model be an ordered state set

$$\mathcal{S} = \{s_1, s_2, \ldots, s_n\}, \qquad n = 9, \tag{1}$$

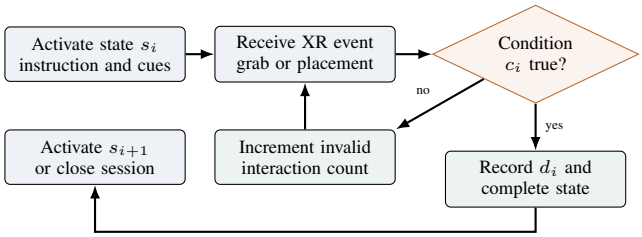


Fig. 4. State transition and interaction-validation logic. Invalid actions are observable but do not advance the procedure.

where a state is represented conceptually as

$$s_i = \langle l_i, a_i, c_i, g_i \rangle. \tag{2}$$

Here, $l_i$ is the displayed instruction, $a_i$ is an optional audio clip, $c_i$ is the completion condition, and $g_i$ identifies associated guidance objects. These fields correspond to the serialized configuration visible in Fig. 2. The state becomes active at time $\tau_i^a$ and completes at $\tau_i^c$, producing duration $d_i = \tau_i^c - \tau_i^a$.

Fig. 4 shows the runtime logic. On activation, the controller updates the instruction panel and cues. A valid interaction completes the state, records its duration, deactivates its cues, and activates the next state. An invalid grab leaves the state unchanged and increments its error counter. The final state closes and exports the session.

### D. Completion Conditions and Guidance

Two condition families are used in the labeling implementation. A grab condition verifies that the expected object has been selected during the active state. A snap-zone condition verifies placement in a designated spatial region. The final label-placement condition also triggers the application animation. Keeping the condition as a configurable state field allows different task objects to reuse the same controller logic.

Guidance is implemented as a policy over the state definition. In guided mode, the active instruction, audio clip, and spatial indicators are enabled. In autonomous mode, the procedure and validators remain active while assistance is suppressed. A perturbed mode can introduce scene constraints without changing the logging contract. Assistance therefore changes what the trainee can perceive, not what counts as a correct action. This separation preserves agency: the system can step back as confidence develops while retaining the same procedure and evidence model.

### E. Telemetry Pipeline

When a session starts, the recorder creates a session context and initializes timers and error counters. Each state transition appends its duration; an interaction involving an unexpected object increments the active state's invalid-grab counter. At completion, the recorder exports the structure

$$R = \langle p, t, m, T, \{d_i\}_1^n, \{e_i\}_1^n \rangle, \tag{3}$$

where $p$ is a session identifier, $t$ is the session timestamp, $m$ is the scene or training mode, $T$ is total duration, and $e_i$ is the invalid-interaction count for state $i$.

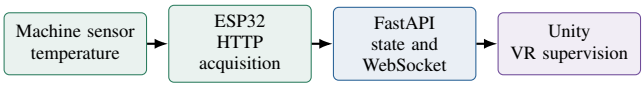


Fig. 5. Implemented supervision proof of concept from a physical measurement to the immersive interface.

The runtime CSV contains operational acquisition metadata, total duration, nine step-duration fields, and nine false-grab fields. The analysis copy excludes names, demographic attributes, and exact timestamps, and replaces identity fields with generated session identifiers. The confidential source file remains separate. All participants provided written informed consent.

## V. Digital-Twin-Ready Supervision

### A. Communication Path

The connected component validates a physical-to-digital data path using a machine-temperature measurement. An ESP32 acquires the value and sends it over the local wireless network. FastAPI was selected as a small, asynchronous mediation service between embedded acquisition and the Unity client. This boundary centralizes message validation and prevents Unity from depending on the ESP32 data format. HTTP provides a simple ingestion and snapshot interface, while WebSocket supports pushed updates without repeated polling. Unity therefore consumes one stable application interface even if the sensor-side implementation changes.

### B. Scope and Failure Boundary

This connection is deliberately separated from the labeling trainer. The training scenario must continue to operate if the network is disconnected, whereas connected supervision must distinguish a fresh measurement from an absent or stale value. The current proof of concept is unidirectional and does not send control commands to production equipment. It is therefore a foundation for a fuller digital twin, not evidence of a bidirectional operational twin.

The boundary also limits safety risk. FastAPI mediates data formats and communication, while Unity remains a visualization and training client. No path from an XR interaction to a machine actuator is provided.

## VI. Technical Evaluation

### A. Prototype Configuration

The prototype runs in Unity 6 with the XR Interaction Toolkit, an XR Origin, and an XR Interaction Manager, and targets a Meta Quest 3 headset. The labeling scene includes interactive bins, bottles, labels, placement zones, world-space instruction panels, arrows, audio guidance, a timer, and a result view. Fig. 1 shows the trainee's workspace, while Fig. 2 exposes the editor-side procedure configuration.

The scenario was exercised in guided, autonomous, and perturbed configurations. These modes use the same state identifiers and recorder. Consequently, changing the guidance policy does not require separate logging code or a different data schema.

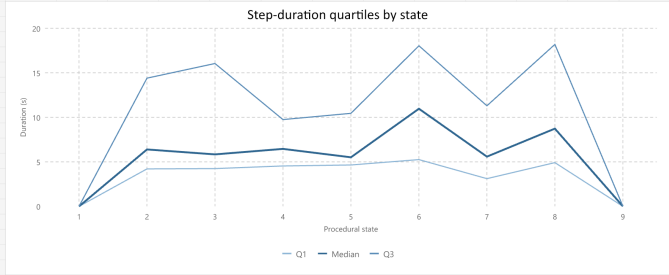


Fig. 6. State-duration telemetry from 27 completed sessions: first quartile, median, and third quartile by procedural state. Boundary states 1 and 9 have near-zero recorded duration.

### B. Measured Results

The evaluation uses 27 completed runs to test the instrumentation path rather than training effectiveness. Every retained record contains the scene identifier, total duration, nine state-duration fields, and nine state-level error counters, giving 100% field completeness for the defined telemetry schema. Across the records, the validator captured 104 invalid-interaction events in 25 sessions. This confirms that rejected actions remained observable without preventing completion.

Recorded session duration ranged from 19.097 to 180.286 s, with a mean of 73.380 s and a median of 61.203 s. More importantly for system verification, the mean absolute difference between total duration and the sum of state durations was 0.445 s; the maximum was 5.999 s. Inspection associated this gap with boundary timing outside state transitions. The result identifies a concrete instrumentation limit: session start and termination must be represented as explicit events when exact time reconciliation is required. No hypothesis test, condition comparison, or learning-effect claim is made.

Fig. 6 resolves the same records by procedural state. States 6 and 8 show the largest upper quartiles for duration. Invalid interactions are concentrated in states 3, 5, and 6, which account for 88 of 104 events (84.6%). The concentration confirms that the logger distinguishes state-specific execution patterns rather than producing only a session total. It also identifies these validators as priorities for technical inspection; it is not interpreted as evidence of participant learning.

### C. Architectural Value

The demonstration supports three technical observations. First, the serialized state list makes the procedure inspectable instead of hiding sequencing entirely in scene callbacks. Second, validators provide a stable semantic boundary between generic XR interaction and task-specific correctness. Third, telemetry is generated by the same transitions that control the procedure, creating a direct chain from requirement to action, validation, and evidence.

## VII. Discussion

### A. Scientific and Engineering Implications

The architecture makes procedural training inspectable at the system level. A scenario can be checked for whether every

state has a completion rule, every invalid action has a defined consequence, and every transition has a corresponding record. The XR rig, recorder, guidance service, and condition types are separated from task-specific scene objects and state sequences.

The communication boundary supports incremental digital-twin development because Unity consumes typed updates rather than communicating directly with embedded hardware. Network latency, packet loss, reconnection time, headset energy cost, and stale-data presentation were not benchmarked in this proof of concept.

### B. Human-Centered Implications

The trainee is not modeled as an unreliable component to be constrained. Invalid grabs are recorded without terminating the task, guidance can be withdrawn without changing correctness rules, and network loss does not remove access to practice. These choices turn mistakes into diagnostic events while preserving continuity and dignity. Trainer-facing records can identify where support may be needed, but they should remain pseudonymous and should complement, not replace, human judgment.

### C. Limitations

The architecture has been evaluated for one manual workstation, so transfer to maintenance or safety procedures remains unverified. The logger is session-oriented rather than an immutable event stream, and the maximum timing gap shows that boundary events require stronger instrumentation. The connected pathway is unidirectional, and its network performance was not measured. Finally, the 27 records demonstrate observability only; they do not show that this architecture improves learning relative to another training method.

## VIII. Conclusion

This paper presented a human-centered, instrumented VR architecture for smart-factory procedures. The Unity 6 and Meta Quest 3 artifact connects six operator-facing actions to nine internal states, explicit validators, graduated guidance, and structured evidence. The evaluation produced complete records for 27 sessions, captured 104 invalid events, and measured a 0.445 s mean absolute timing gap. A separated ESP32–FastAPI–Unity pathway introduced physical context without making practice dependent on network availability or exposing equipment to VR-originated control. The contribution is a modular platform in which procedural semantics make execution measurable while recoverable errors and adjustable guidance keep the trainee active.

## References


[1] D. Jones, C. Snider, A. Nassehi, J. Yon, and B. Hicks, "Characterising the digital twin: A systematic literature review," *CIRP Journal of Manufacturing Science and Technology*, vol. 29, pp. 36–52, 2020.

[2] A. Rasheed, O. San, and T. Kvamsdal, "Digital twin: Values, challenges and enablers from a modeling perspective," *IEEE Access*, vol. 8, pp. 21 980–22 012, 2020.

[3] S. Doolani, C. Wessels, V. Kanal, C. Sevastopoulos, A. Jaiswal, H. Nambiappan, and F. Makedon, "A review of extended reality (XR) technologies for manufacturing training," *Technologies*, vol. 8, no. 4, p. 77, 2020.

[4] S. Garcia Fracaro, J. Glassey, K. Bernaerts, and M. Wilk, "Immersive technologies for the training of operators in the process industry: A systematic literature review," *Computers & Chemical Engineering*, vol. 160, p. 107691, 2022.

[5] J. Radianti, T. A. Majchrzak, J. Fromm, and I. Wohlgenannt, "A systematic review of immersive virtual reality applications for higher education: Design elements, lessons learned, and research agenda," *Computers & Education*, vol. 147, p. 103778, 2020.

[6] M. Hu, X. Luo, J. Chen, Y. C. Lee, Y. Zhou, and D. Wu, "Virtual reality: A survey of enabling technologies and its applications in IoT," arXiv:2103.06472, 2021. [Online]. Available: https://arxiv.org/abs/2103.06472

[7] P. Strojny and N. Duzmanska-Misiarczyk, "Measuring the effectiveness of virtual training: A systematic review," *Computers & Education: X Reality*, vol. 2, p. 100006, 2023.

[8] M. E. Latoschik and C. Wienrich, "Congruence and plausibility, not presence?! pivotal conditions for XR experiences and effects, a novel model," *Frontiers in Virtual Reality*, vol. 2, 2021.

[9] A. Fuller, Z. Fan, C. Day, and C. Barlow, "Digital twin: Enabling technologies, challenges and open research," *IEEE Access*, vol. 8, pp. 108 952–108 971, 2020.

[10] A. Thelen, X. Zhang, O. Fink, Y. Lu, S. Ghosh, B. D. Youn, M. D. Todd, S. Mahadevan, C. Hu, and Z. Hu, "A comprehensive review of digital twin–part 1: Modeling and twinning enabling technologies," *Structural and Multidisciplinary Optimization*, vol. 65, no. 12, p. 354, 2022.

[11] J. Wilhelm, C. Petzoldt, T. Beinke, and M. Freitag, "Review of digital twin-based interaction in smart manufacturing: Enabling cyber-physical systems for human-machine interaction," *International Journal of Computer Integrated Manufacturing*, vol. 34, no. 10, pp. 1031–1048, 2021.

[12] *Automation Systems and Integration–Digital Twin Framework for Manufacturing–Part 1: Overview and General Principles*, International Organization for Standardization Std. ISO 23 247-1:2021, 2021.

[13] Y. Lu, C. Liu, K. I.-K. Wang, H. Huang, and X. Xu, "Digital twin-driven smart manufacturing: Connotation, reference model, applications and research issues," *Robotics and Computer-Integrated Manufacturing*, vol. 61, p. 101837, 2020.

[14] F. Longo, A. Padovano, and S. Umbrello, "Value-oriented and ethical technology engineering in industry 5.0: A human-centric perspective for the design of the factory of the future," *Applied Sciences*, vol. 10, no. 12, p. 4182, 2020.

[15] Y. Lu, H. Zheng, S. Chand, W. Xia, Z. Liu, X. Xu, L. Wang, Z. Qin, and J. Bao, "Outlook on human-centric manufacturing towards industry 5.0," *Journal of Manufacturing Systems*, vol. 62, pp. 612–627, 2022.

[16] B. Wang, P. Zheng, Y. Yin, A. Shih, and L. Wang, "Toward human-centric smart manufacturing: A human-cyber-physical systems perspective," *Journal of Manufacturing Systems*, vol. 63, pp. 471–490, 2022.

[17] J. Alves, T. M. Lima, and P. D. Gaspar, "Is industry 5.0 a human-centred approach? a systematic review," *Processes*, vol. 11, no. 1, p. 193, 2023.

[18] X. Li, A. Nassehi, B. Wang, S. J. Hu, and B. I. Epureanu, "Human-centric manufacturing for human-system coevolution in industry 5.0," *CIRP Annals*, vol. 72, no. 1, pp. 393–396, 2023.

[19] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[20] Khronos Group, "OpenXR: High-performance access to AR and VR platforms and devices," Open standard, 2024. [Online]. Available: https://www.khronos.org/openxr/